tty 运行时调用链（从用户态 / 中断入口一直到 Mini UART 寄存器）

A. read(0, buf, n)

sys_read
argfd → struct file
fileread(f, addr, n)
fs/sysfile.c:751

↓

fileread
f->f_op->fread? 否
f->f_op->read(1, addr, n)
fs/file.c:166

↓

ttycurrentread（若经 /dev/tty）
p->ctty != TTYS0 → -1
return consoleread(...)
tty.c:18

↓

consoleread
🔒 acquire(cons.lock)
while r == w:
killed → 放锁 return -1
sleep(&cons.r, &cons.lock)
c = buf[r++]，^D 处理
either_copyout(…, 1 字节)
遇 \n 退出；release
console.c:152

↓

sleep(&cons.r, lk)
acquire(p->lock); release(lk)
state=SLEEPING; sched()
↑ 被 consoleintr 的 wakeup 唤醒
core/proc.c:1193

wakeup 唤醒 A 列的 sleep

B. 串口按键 → 中断

异常向量
EL0: bl userirq
EL1: bl kernelirq
arch/arm64/trapasm.S

↓

devintr
irq = gic_iar()
irq == UART0_IRQ(29)
→ uartintr()
core/trap.c:163

↓

uartintr
while (c = uartgetc()) != -1:
consoleintr(c)
🔒 uart_tx_lock: uartstart()
uart.c:212

↓

consoleintr(c)
🔒 acquire(cons.lock)
^H ^U ^P ^C 特判
consputc(c) 回显
buf[e++] = c
\n/^D/满 → w=e
wakeup(&cons.r)
release(cons.lock)
console.c:208

↓

wakeup(&cons.r)
遍历 proc_head
🔒 p->lock: SLEEPING &&
chan 相同 → make_runnable
core/proc.c:1224

C. write(1, buf, n)

sys_write → filewrite
f->f_op->write(1, addr, n)
fs/file.c:227

↓

consolewrite
for each byte:
either_copyin(&c, …, 1)
\n 前补 uartputc('\r')
uartputc(c)
无 cons.lock，无写者互斥
console.c:129

↓

uartputc(c)
🔒 acquire(uart_tx_lock)
环满 → sleep(&uart_tx_r)
tx_buf[w++] = c
uartstart(); release
uart.c:124

↓

uartstart
while tx_r != tx_w:
uart_tx_try(c) 失败 →
开 TX 中断，return
tx_r++; wakeup(&uart_tx_r)
空 → 关 TX 中断
uart.c:174

↓

uart_tx_try
🔒 uart_hw_lock
mu_lsr & TX_SPACE ?
mu_io = c
uart.c:57

D. printf / 回显

printf(fmt, …)
🔒 pr.lock（pr.locking 时）
逐字符 consputc
lib/printf.c:64

↓

consputc(c)
BACKSPACE → \b 空格 \b
\n → 先发 \r
uartputc_sync(c)
console.c:35

↓

uartputc_sync(c)
push_off()（关本核中断）
panicked → 原地死循环
while !uart_tx_try(c) 忙等
pop_off()
uart.c:153

↓

uart_tx_try
🔒 uart_hw_lock
写 mu_io
uart.c:57

↓

注意
不进 32 B 环、不睡眠
可在中断里调用（回显就是）
可能插队到排队中的 write 字节前面

E. ^C → SIGINT

consoleintr(C('C'))
🔒 cons.lock
r = e = w 清空当前行
回显 "^C\n"
wakeup(&cons.r)
release(cons.lock)
ttyintr() ← 放锁之后
console.c:213

↓

ttyintr
🔒 ttyfg_lock
pgid = ttyfg_pgid
release
signal_pgrp(pgid, SIGINT)
tty.c:86

↓

signal_pgrp
for p in proc_head:
🔒 p->lock
pgid 匹配 && ctty==TTYS0:
signals_pending |= …
killed = 1
SLEEPING → make_runnable
core/proc.c:1266

↓

前台进程醒来
consoleread 看到 killed
→ return -1
回用户态前 usertrap 处理退出

USB 键盘也走 B 列的后半段

DWC2_IRQ → devintr → dwc2_irq() → URB 完成 → usbkbd_irq_complete（🔒kbd->lock，只 queue_work）→ kworker 线程 → usbkbd_rx_work
→ usbkbd_key()：HID 码查 keymap/shiftmap，Ctrl 变控制字符，方向键 emit_escape(ESC [A/B/C/D) → consoleintr(c) （此时已放掉 kbd->lock）
→ 之后与串口完全相同：cons.lock、回显、wakeup(&cons.r)、^C。另外同一事件还经 input_report_key 发给 evdev（/dev/input/event0），两路互不依赖。

颜色：蓝=输入环 cons，绿=write 路径，橙=同步输出，紫=tty 会话层，红=中断入口。🔒 = spinlock（acquire 时关本核中断）。